

Aplicações e Serviços Web 2025/2026

TP8: Integração de Serviços e Comunicação entre Sistemas

1. Instalação e Configuração

Neste guião o fetch será a nossa ferramenta principal para interagir com serviços externos (biblioteca nativa do node.js). Vamos precisar de instalar as restantes bibliotecas necessárias.

Nota: Para este guião irá necessitar de ter todas as ferramentas dos guiões anteriores já instaladas. Pode ser útil copiar os ficheiros package.json e package-lock.json e de seguida correr o npm install caso esteja a usar uma nova pasta para este guião, desta forma todos os package irão ser instalados.

1. De seguida deve executar os seguintes comandos no terminal para instalar as ferramentas necessárias:
npm install multer node-cache
npm install -D @types/multer

2. Consumo de APIs Externas

Vamos explorar o acesso a APIs externas com dois exemplos. Note que nem sempre é necessário guardar os dados das APIs externas na nossa base de dados e podemos apenas usar os dados obtidos através da API externa como resposta aos pedidos efectuados à nossa API.

2.1 Acesso a Dados Geográficos (Rest Countries)

Vamos criar um serviço que consome a API [Rest Countries](#) (sem necessidade de chave) para validar ou obter dados de países. O recurso País não é persistido na nossa base de dados existindo apenas como uma interface de dados que consumimos da Rest Countries.

2. Vamos começar por criar o código necessário para criar o serviço que irá comunicar com esta api externa, para isso vamos criar o ficheiro **countryService.ts** e adicionar o seguinte pedaço de código:

```
export interface Country {  
  name: string;  
  capital: string;  
  region: string;  
  flag: string;  
}
```

```

export const getCountryByCode = async (code: string): Promise<Country | null> => {
  // Usamos o endpoint /alpha/{code} para códigos de 2 ou 3 letras (ex: PT, PRT)
  const response = await fetch(`https://restcountries.com/v3.1/alpha/${code}`);

  if (response.status === 404) return null;
  if (!response.ok) throw new Error("Falha ao comunicar com API de Países");

  const data = await response.json();
  const c = data[0];

  return {
    name: c.name.common,
    capital: c.capital?.[0] || "N/A",
    region: c.region,
    flag: c.flags.svg
  };
};

```

Nota: Como o fetch é uma operação assíncrona, a função não devolve o objeto País imediatamente, mas sim uma **Promise** de que eventualmente devolverá um objeto do tipo Country ou null (se não encontrar). O async funciona como um contrato automático onde não é necessário criar a Promise manualmente com new Promise, o JS faz essa gestão no return. O que ocorre automaticamente é equivalente a return Promise.resolve({ name: "Portugal", capital: "Lisboa", ... });

3. Deve criar um countryController.ts que invoque o serviço e um countryRoutes.ts que defina a rota GET /api/countries/:code, bem como documentar e validar os parâmetros da rota de forma adequada.
4. No terminal, execute o script para ligar o servidor:
npx tsx watch src/server.ts
5. Realize vários pedidos de forma a verificar se consegue aceder ao serviço externo, nomeadamente varie o código para valores válidos e inválidos.

2.2 Acesso a Dados de Livros (Google Books APIs)

Vamos criar/modificar o recurso Book para que passe a conter um isbn que nos permita ir buscar informação adicional sobre o mesmo a uma api externa. Para isso vamos criar um serviço que consome a API do [Google Books](https://books.google.com/books).

1. Vamos começar por criar o código necessário para criar o serviço que irá comunicar com esta api externa, para isso vamos criar o ficheiro **externalBookService.ts** e adicionar o seguinte pedaço de código:

```

export const getExternalDescription = async (isbn: string): Promise<string | null> => {
  const response = await
fetch(`https://www.googleapis.com/books/v1/volumes?q=isbn:${isbn}`);
  if (!response.ok) return null;

  const data = await response.json();
  return data.items?.[0]?.volumeInfo?.description || null;
};

```

2. Agora vamos atualizar ou criar um novo **bookService.ts** para que dado um isbn possa interagir com o externalBookService.ts para ir buscar a descrição do livro se disponível, para isso devemos adicionar atualizar o método de createBook:

```

import * as externalBookService from './externalBookService.js';

export const createBook = async (bookData: { title: string; isbn: string }) => {
  // 1. Tentar obter descrição externa
  const description = await
externalBookService.getExternalDescription(bookData.isbn);

  // 2. Regra de Negócio: Se não existir descrição na Google, o registo é rejeitado
  if (!description) {
    throw new Error("Não foi possível encontrar uma descrição para este ISBN. Registo cancelado.");
  }

  return await prisma.book.create({
    data: { ...bookData, description }
  });
};

```

3. Deve criar um model Book para que passe a conter o isbn e description:

```

model Book {
  id      Int    @id @default(autoincrement())
  title   String
  isbn    String @unique // Necessário para a consulta na Google Books
  description String?    // Campo opcional para guardar a sinopse
  createdAt DateTime @default(now())
}

```

Nota: Não se esqueça de executar os comandos prisma generate e migrate para que alterações sejam refletidas

4. Deve criar um **bookController.ts** que suporte a rota de criação de um novo livro. Note que se reutilizar o seu bookController.ts de guiões anteriores terá de atualizar os schema de validação e os ficheiros de documentação de forma a suportar as alterações efetuadas para a rota de criação de um novo livro.
5. No terminal, se necessário volte a executar o script para ligar o servidor:
npx tsx watch src/server.ts
6. Realize vários pedidos de forma a verificar se consegue aceder ao serviço externo, nomeadamente varie o isbn para valores válidos e inválidos.

3. Gestão de Ficheiros e Upload de Imagens

Agora vamos explorar como podemos gerir o armazenamento de ficheiros no nosso servidor, focando-nos em suporte para ficheiros multimédia (ex: imagens). Para este efeito, utilizaremos a biblioteca **multer e fs**.

É importante notar que, embora seja tecnicamente possível guardar imagens diretamente na base de dados como BLOB, esta prática deve ser evitada. O armazenamento de ficheiros na base de dados aumenta drasticamente o tamanho dos backups, degrada a performance das consultas (queries) e consome recursos de memória do servidor de base de dados que deveriam estar otimizados para dados estruturados. Em vez disso, a norma recomendada é guardar os ficheiros de forma independente: ou no sistema de ficheiros local do servidor ou em serviços de armazenamento na cloud (como AWS S3 ou Google Cloud Storage) para maior escalabilidade. Na base de dados, guardamos apenas a referência (o caminho ou URL) para esses ficheiros, mantendo a aplicação leve, rápida e fácil de manter.

1. Vamos começar por criar o código necessário para suportar as funções necessárias para guardar e realizar o upload de um ficheiro. Para que possam ser reutilizadas iremos criar um ficheiro **lib/multer.ts** para acomodar estes métodos e adicionar o seguinte pedaço de código:

```
import multer from 'multer';
import path from 'path';

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, `${file.fieldname}-${uniqueSuffix}${path.extname(file.originalname)}`);
  }
});
```

```
export const upload = multer({
  storage,
  limits: { fileSize: 2 * 1024 * 1024 } // Limite de 2MB
});
```

Nota: De forma a podermos configurar de forma ágil a pasta onde são colocados os ficheiros poderíamos definir o seu nome no ficheiro .env

2. Vamos agora criar o middleware necessário para suportar a opção de upload de imagens que pode ser utilizado nas rotas quando necessário, para isso devemos criar o ficheiro **middlewares/uploadMiddleware.ts** e adicionar o seguinte pedaço de código:

```
import { upload } from '../lib/multer.js';

export const uploadBookCover = upload.single('cover');
```

Nota: Desta forma caso seja necessário alterar a biblioteca de upload as rotas não necessitam de ser alteradas.

3. Agora vamos criar o código necessário para eliminar ficheiros existentes quando formos atualizar o caminho (coverUrl) de um Book. Para isso vamos criar o código necessário para suportar as funções necessárias para eliminar um ficheiro. Para que possam ser reutilizadas iremos criar um ficheiro **lib/fileStorage.ts** para acomodar estes métodos e adicionar o seguinte pedaço de código:

```
import fs from 'fs/promises';
import path from 'path';

export const deleteFile = async (filePath: string | null) => {
  if (!filePath) return;
  try {
    await fs.unlink(path.resolve(filePath));
  } catch (error) {
    console.error(`Erro ao apagar ficheiro: ${filePath}`, error);
  }
};
```

4. Deve atualizar o model Book (criado em 2) para que passe a conter o coverUrl que guarda o caminho para as imagens de capa do livro:

coverUrl String? // Caminho do ficheiro (ex: "uploads/cover-xxx.jpg")

Nota: Não se esqueça de executar os comandos prisma generate e migrate para que alterações sejam refletidas

5. Vamos agora atualizar o **bookService.ts** para passar a suportar as operações de upload e download da capa do livro, para isso vamos adicionar o seguinte pedaço de código:

```
import { prisma } from '../lib/prisma.js';
import { deleteFile } from '../lib/fileStorage.js';
import path from 'path';
import fs from 'fs';

export const updateBookCover = async (bookId: number, newPath: string) => {
  const book = await prisma.book.findUnique({ where: { id: bookId } });
  if (!book) throw new Error("Livro não encontrado");

  if (book.coverUrl) await deleteFile(book.coverUrl);

  return await prisma.book.update({
    where: { id: bookId },
    data: { coverUrl: newPath }
  });
};

export const getBookCoverAbsolutePath = async (bookId: number) => {
  const book = await prisma.book.findUnique({ where: { id: bookId } });
  if (!book || !book.coverUrl) return null;

  const absolutePath = path.resolve(book.coverUrl);
  return fs.existsSync(absolutePath) ? absolutePath : null;
};
```

6. Vamos agora adicionar no **bookController.ts** as duas operações necessárias para suportar o upload e download de capas de livros, acrescentado o seguinte pedaço de código:

```
import * as bookService from '../services/bookService.js';

export const uploadCover = async (req: Request, res: Response) => {
  try {
```

```

        const updated = await bookService.updateBookCover(Number(req.params.id),
req.file!.path);
        res.json(updated);
    } catch (error: any) {
        res.status(404).json({ error: error.message });
    }
};

export const downloadCover = async (req: Request, res: Response) => {
    try {
        const filePath = await
bookService.getBookCoverAbsolutePath(Number(req.params.id));
        if (!filePath) return res.status(404).json({ error: "Capa não encontrada" });

        res.sendFile(filePath);
    } catch (error) {
        res.status(500).json({ error: "Erro no servidor" });
    }
};

```

7. Finalmente precisamos de adicionar as novas rotas para suportar a operações de upload e download de capas de livros, para isso vamos acrescentar o seguinte pedaço de código ao ficheiro **bookRoutes.ts**:

```

import { Router } from 'express';
import * as bookController from '../controllers/bookController.js';
import { uploadBookCover } from '../middlewares/uploadMiddleware.js';

const router = Router();
router.put('/:id/cover', uploadBookCover, bookController.uploadCover);
router.get('/:id/cover', bookController.downloadCover);

export default router;

```

8. No terminal, se necessário volte a executar o script para ligar o servidor:
npm run dev
9. Realize vários pedidos de forma a verificar se consegue realizar upload e download de capas para livro. Verifique se os ficheiros foram corretamente adicionados à pasta respectiva do servidor e se o url se encontra guardado na base de dados.

4. Integração com Google OAuth

Vamos explorar como podemos usar serviços externos para gerir as credenciais dos utilizadores através do uso do Google OAuth. Como não temos Frontend, utilizaremos o Google OAuth Playground para simular o utilizador e obter um código real para testar a nossa API.

4.1 Obter um Código de Teste (Google OAuth Playground)

1. Aceda ao [Google OAuth Playground](#).
2. No canto superior direito clique na roda dentada (configurações) e marque a opção Use your own OAuth credentials
3. Preencha o OAuth Client ID e OAuth Client secret (ver nota)
4. No passo 1 (Select & authorize APIs), procure por Google OAuth2 API v2 e selecione os scopes **email, profile, openid**.
5. Clique em Authorize APIs e faça login com a sua conta Google.
6. No passo 2, copie o **Authorization code**. É este código que enviará no corpo do pedido à API.

Nota: Para obter os seus próprios valores para OAuth Client ID e OAuth Client secret deverá seguir os passos descritos neste [tutorial](#)

Em alternativa facilitar o uso da biblioteca pode usar os seguintes valores para client id e secret:

GOOGLE_CLIENT_ID: 979548221087-g24oprne6f24v205ncuvbr5ftrrhoksp.apps.googleusercontent.com

GOOGLE_CLIENT_SECRET: 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

4.2 Suportar autenticação GoogleOAuth na API

Vamos agora criar a estrutura necessária para suportar na nossa API a verificação de tokens fornecidos pelo Google OAuth.

1. Para isso vamos começar por criar um novo serviço no ficheiro **services/googleAuthService.ts** e adicionar o seguinte pedaço de código:

```
import { jwtDecode } from 'jwt-decode';
```

```
export const verifyGoogleCode = async (code: string) => {  
  // 1. Troca do Code por Tokens  
  const response = await fetch('https://oauth2.googleapis.com/token', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({  
      code,  
      client_id: process.env.GOOGLE_CLIENT_ID,
```



```

    client_secret: process.env.GOOGLE_CLIENT_SECRET,
    redirect_uri: process.env.GOOGLE_REDIRECT_URI,
    grant_type: 'authorization_code',
  }},
});

if (!response.ok) throw new Error("Falha na autenticação com Google");

const data = await response.json();

// 2. Decodificar ID Token (JWT) para obter identidade
// Retorna { email, name, picture, sub }
return jwtDecode(data.id_token) as any;
};

```

Nota: Para obter os seus próprios valores para `GOOGLE_CLIENT_ID`, e `GOOGLE_CLIENT_SECRET` deverá seguir os passos descritos neste [tutorial](#). Em alternativa facilitar o uso da biblioteca pode usar os seguintes valores para `client_id`, `secret` e `redirect_uri`:

GOOGLE_CLIENT_ID: 979548221087-g24oprne6f24v205ncuvbr5ftrrhoksp.apps.googleusercontent.com

GOOGLE_CLIENT_SECRET: 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

GOOGLE_CLIENT_REDIRECT_URI: <https://developers.google.com/oauthplayground>

Nota2: O **Authorization code** criado em 4.1 é apenas válido para um pedido (mesmo que o mesmo não tenha tido sucesso ou dê erro), pelo que terá de gerar novo código em cada pedido.

- De seguida no **controllers/authController.ts** vamos adicionar um método que permite receber e processa os pedidos com login da google, para isso vamos adicionar o seguinte pedaço de código:

```

import * as googleAuthService from '../services/googleAuthService.js';
import * as userService from '../services/userService.js';
import jwt from 'jsonwebtoken';

export const googleLogin = async (req: Request, res: Response) => {
  try {
    const { code } = req.body;

    // Delegar a validação ao serviço
    const googleUser = await googleAuthService.verifyGoogleCode(code);

    // Lógica de negócio: encontrar ou criar utilizador

```

```

let user = await userService.findUserByEmail(googleUser.email);
if (!user) {
  user = await userService.createUser({
    email: googleUser.email,
    fullName: googleUser.name
  });
}

const token = jwt.sign({ id: user.id }, process.env.JWT_SECRET!);
res.json({ token, user: googleUser });
} catch (error) {
  res.status(401).json({ error: "Login social falhou" });
}
};

```

3. Deve adicionar a rota (/auth/google) para suportar o login através do GoogleOAuth. O modelo User de guiões anteriores pode ser reutilizado devendo o campo password passar a ser opcional, em alternativa pode criar um novo modelo para representar um utilizador que contenha os campos email e fullName. Pode reutilizar o userService de guiões anteriores para suportar os métodos findUserByEmail e createUser, em alternativa pode também criar um novo **userService.ts** adicionar o seguinte pedaço de código:

```

import { prisma } from "../lib/prisma.js";

export const createUser = async (fullName: string, email: string) => {

  return await prisma.user.create({
    data: { fullName, email },
  });
};

export const findUserByEmail = async (email: string) => {
  return await prisma.user.findUnique({
    where: { email }
  });
}

```

4. Deve criar um countryController.ts que invoque o serviço e um countryRoutes.ts que defina a rota GET /api/countries/:code, bem como documentar e validar os parâmetros da rota de forma adequada.
5. No terminal, se necessário volte a executar o script para ligar o servidor:
npx tsx watch src/server.ts

6. Realize vários pedidos de forma a verificar se consegue realizar um login e obter um token válido através do uso do GoogleOAuth.

5. Integração com LLMs

Vamos explorar como podemos usar serviços inteligentes dentro de uma aplicação web. Para garantir a modularidade e evitar o acoplamento a um serviço específico (e.g., OpenAI, Ollama), utilizaremos o padrão *Strategy* para o consumo de serviços inteligentes (LLMs).

5.1 Interface de Abstração

1. Vamos começar por criar uma interface (**src/services/llm/ILLMProvider.ts**) para suportar a representação de um LLMProvider, para isso vamos acrescentar o seguinte pedaço de código:

```
export interface ILLMProvider {  
  generateResponse(prompt: string): Promise<string>;  
}
```

5.2 Mock de Simulação para Interação

1. Vamos agora criar um mock que simula a interação com uma LLM, que será suportada por uma gerador aleatório que simula o não determinismo de um modelo. Vamos adicionar ao ficheiro **src/services/llm/MockLLMProvider.ts** o seguinte pedaço de código:

```
import { ILLMProvider } from './ILLMProvider.js';  
  
export class MockLLMProvider implements ILLMProvider {  
  async generateResponse(prompt: string): Promise<string> {  
    const adjetivos = ["fascinante", "complexo", "intrigante", "melancólico"];  
    const conclusoes = ["uma obra prima.", "um marco da literatura.", "altamente recomendado."];  
  
    const randomAdj = adjetivos[Math.floor(Math.random() * adjetivos.length)];  
    const randomConc = conclusoes[Math.floor(Math.random() * conclusoes.length)];  
  
    return `[MOCK AI]: Analisando o prompt "${prompt.substring(0, 20)}...", considero este tema ${randomAdj} e, em suma, é ${randomConc}`;  
  }  
}
```

5.3 Classe Genérica para LLM API

1. Vamos agora criar uma classe genérica que poderá ser utilizada para suportar qualquer acesso a uma API LLM. Vamos adicionar ao ficheiro **src/services/llm/GenericAPIProvider.ts** o seguinte pedaço de código:

```

import { ILLMProvider } from "./ILLMProvider.js";

export class GenericAPIProvider implements ILLMProvider {
  constructor(
    private apiUrl: string,
    private modelName: string,
    private apiKey?: string, // Opcional para APIs locais (ex: Ollama)
    private inputField = "prompt" // "prompt", "inputs" ou "messages"
  ) {}

  async generateResponse(prompt: string): Promise<string> {
    const headers: Record<string, string> = { "Content-Type": "application/json" };

    if (this.apiKey) headers["Authorization"] = `Bearer ${this.apiKey}`;

    // Montagem genérica: se o campo for 'messages', encapsula o prompt num array
    const inputContent = this.inputField === "messages"
      ? [{ role: "user", content: prompt }]
      : prompt;

    const res = await fetch(this.apiUrl, {
      method: "POST",
      headers,
      body: JSON.stringify({
        model: this.modelName,
        [this.inputField]: inputContent,
        stream: false,
      }),
    });

    if (!res.ok) throw new Error(`Erro API (${res.status}): ${await res.text()}`);

    const data = await res.json();

    // Extração genérica: tenta todos os caminhos comuns de resposta
    return (
      data.choices?.[0]?.message?.content || // Padrão OpenAI / HF Router
      (Array.isArray(data) ? data[0]?.generated_text : null) || // Padrão HF Inference
      data.response || // Padrão Ollama
      data.output || // Outros
      "Sem resposta."
    );
  }
}

```

```
);  
}  
}
```

5.4 Configuração de APIs para Integração de LLMs

2. Vamos explorar como instalar e configurar o **Ollama** (uma LLM que corre localmente no nosso computador. Para isso deve seguir estes passos:
 - a. Aceda a ollama.com e instale a versão correspondente ao seu SO.
 - b. No terminal, execute o comando: `ollama run llama3` (ou outro modelo como `mistral`).
 - c. O Ollama disponibiliza por defeito uma API REST no endereço <http://localhost:11434/api/generate>, aceda a esse endereço e explore realizar alguns pedidos
3. Vamos explorar como usa APIs de LLM remotas. Embora a maioria dos serviços (OpenAI, Anthropic) seja paga, existem alternativas para fins académicos. Para isso deve seguir estes passos para o uso da Hugging Face API:
 - a. Crie uma conta em huggingface.co.
 - b. Vá às definições do seu perfil (Settings) > Access Tokens.
 - c. Clique em New Token, dê um nome (ex: "TP8-API") e selecione o tipo Read.
 - d. Copie o token gerado e adicione-o ao seu ficheiro .env:
`HF_TOKEN=hf_XXXXXXXXXXXXXXXXXX`

5.5 Integração da LLM na API

Vamos criar a estrutura necessária para que a nossa API passe a suportar serviços inteligentes. Para demonstrar a flexibilidade da nossa arquitetura, vamos implementar três endpoints. Cada um utilizará o mesmo serviço de negócio, mas injetará um provider de LLM diferente.

1. Vamos criar ou acrescentar ao serviço do book (**src/services/bookService.ts**) o pedaço de código abaixo para suportar a extração de um sumário de um livro através de um modelo LLM:

```
import { prisma } from '../lib/prisma.js';  
import { ILLMProvider } from '../llm/ILLMProvider.js';  
  
export const generateBookSummary = async (id: number, llm: ILLMProvider) =>  
{  
  // 1. Pesquisa por ID na base de dados local  
  const book = await prisma.book.findUnique({ where: { id } });  
  
  if (!book) throw new Error("Livro não encontrado no catálogo.");
```

```

    // 2. Construção do prompt utilizando os dados persistidos (Título e
    Descrição)
    const prompt = `Escreve um resumo conciso e profissional do livro
    "${book.title}".
    Contexto: ${book.description || 'Sem descrição disponível.'}`;

    // 3. Delegação da tarefa ao Provider injetado
    return await llm.generateResponse(prompt);
  };

```

2. Vamos agora criar ou acrescentar ao controlador do livro (**src/controllers/bookController.ts**) os pedaços de código abaixo que servirão de suporte as 3 rotas que iremos criar:

```

import { Request, Response } from 'express';
import * as bookService from '../services/bookService.js';
import { MockLLMProvider } from '../services/llm/MockLLMProvider.js';
import { GenericAPIProvider } from '../services/llm/GenericAPIProvider.js';

// 1. Mock Summary: Simulação local com texto aleatório
export const getMockSummary = async (req: Request, res: Response) => {
  const llm = new MockLLMProvider();
  try {
    const summary = await
    bookService.generateBookSummary(Number(req.params.id), llm);
    res.json({ mode: "mock", summary });
  } catch (error: any) {
    res.status(404).json({ error: error.message });
  }
};

// 2. Local Summary: Utiliza o Ollama a correr na máquina local
export const getLocalSummary = async (req: Request, res: Response) => {
  const llm = new GenericAPIProvider(
    'http://localhost:11434/api/generate',
    'llama3',
    undefined, // Sem API Key para uso local
    'prompt' // Ollama usa o campo 'prompt'
  );

  try {

```

```

        const summary = await
bookService.generateBookSummary(Number(req.params.id), llm);
        res.json({ mode: "local-ollama", summary });
    } catch (error: any) {
        res.status(500).json({ error: "Falha na comunicação com o Ollama local." });
    }
};

```

```

// 3. API Summary: Utiliza o Hugging Face (Cloud)
export const getApiSummary = async (req: Request, res: Response) => {
    const llm = new GenericAPIProvider(
        'https://router.huggingface.co/v1/chat/completions',
        'katanemo/Arch-Router-1.5B:hf-inference',
        process.env.HF_TOKEN, // Requer token HF_TOKEN no .env
        'messages'
    );

    try {
        const summary = await
bookService.generateBookSummary(Number(req.params.id), llm);
        res.json({ mode: "cloud-api", summary });
    } catch (error: any) {
        res.status(502).json({ error: "Erro na resposta da API Cloud (Hugging Face)." });
    }
};

```

3. Vamos criar 3 rotas (**src/routes/bookRoutes.ts**) para suportar os endpoints necessários:

```

import { Router } from 'express';
import * as bookController from '../controllers/bookController.js';

const router = Router();

// Endpoints de Resumo (LLM)
router.get('/:id/mock-summary', bookController.getMockSummary);
router.get('/:id/local-summary', bookController.getLocalSummary);
router.get('/:id/api-summary', bookController.getApiSummary);

export default router;

```

4. No terminal, se necessário volte a executar o script para ligar o servidor:
`npx tsx watch src/server.ts`
5. Realize vários pedidos de forma a experimentar cada um dos endpoints criados. Para o local-summary deverá ter iniciado o ollama no terminal. É possível que neste modelo local o tempo de espera para obter uma resposta seja mais elevado.

6. Exercícios

Para os exercícios abaixo deve criar a estrutura necessária para que uma organização por camadas seja seguida. Portanto deverá criar a rota, controller, service e se aplicável adicionar um modelo. Note que as chamadas a serviços externos devem ter um serviço próprio e deve ser promovida a reutilização de código.

Exercício 1

Crie a rota **GET /api/weather/:city**. O serviço deve consultar a [Geocoding da Open-Meteo](#) para obter a latitude e longitude da cidade e, de seguida, consultar a API de [Forecast](#) para obter a temperatura atual.

Exercício 2

Crie a rota **GET /api/countries/:code/weather**. O serviço deve obter a capital de um país através da API Rest Countries e usar essa capital para obter a previsão do tempo, devendo para isso reutilizar o serviço criado no Exercício 1. Deve devolver um JSON com: Nome do País, Bandeira, Capital e Temperatura.

Exercício 3

Utilize a biblioteca **node-cache** para que a meteorologia de um país seja guardada por 10 minutos. Desta forma são evitadas chamadas desnecessárias às duas APIs externas se o código do país for repetido.

Exercício 4

Adicione um **timeout** de 3 segundos em todos os pedidos fetch. Se a API da Google, Rest Countries ou Open-Meteo demorar a responder, o servidor deve cancelar a operação e retornar um erro 504.

Nota: Pode ser útil utilizar a classe **AbortController**

Exercício 5

Configure o Multer para aceitar apenas imagens (jpeg/png) até 10MB. No controlador de upload da capa do livro, se a atualização na base de dados falhar, use o utilitário `deleteFile` para remover o ficheiro físico que foi gravado.

Exercício 6

Altere a rota **GET /api/countries/:code** para que devolva vários detalhes e curiosidades do país usando para isso a LLM através do serviço criado anteriormente.

7. Tutoriais

Deve realizar os tutoriais de suporte abaixo e consultar a documentação oficial:

- Google OAuth
 - [OpenID Connect Guide](#)
 - [Google OAuth Playground](#)
- APIS
 - [Geocoding API](#)
 - [Weather Forecast API](#)
 - [Rest Countries](#)
 - [Google Books](#)
- Bibliotecas Node
 - [Fetch](#)
 - [Multer](#)
 - [Node-cache](#)
- APIs LLMs
 - [Ollama](#)
 - [Hugging Face](#)
 - [Hugging Face Inference Providers](#)